DOCUMENTACIÓN TÉCNICA

API TechStore

Manual de uso

Administración de una tienda de tecnología con Python, FastAPI y SQL Server

1. Descripción

TechStore usa una API desarrollada con Python, FastAPI y SQL Server para administrar la información de una tienda de tecnología. Permite gestionar productos, categorías, marcas, proveedores y clientes, registrar compras y ventas mediante procedimientos almacenados y consultar reportes de ventas e inventario comercial.

2. Requisitos previos

Antes de ejecutar el proyecto, se necesita:

- Python instalado.
- Microsoft SQL Server.
- SQL Server Management Studio (SSMS).
- Microsoft ODBC Driver 18 for SQL Server.
- La base de datos TechStore creada con sus tablas, relaciones, datos de prueba y procedimientos almacenados.
- Las bibliotecas Python fastapi, uvicorn y pyodbc.

3. Configuración de la conexión

El archivo database.py contiene la función que conecta la API con SQL Server.

```
import pyodbc

def obtener_conexion():
    conexion = pyodbc.connect(
        "DRIVER={ODBC Driver 18 for SQL Server};"
        "SERVER=localhost;"
        "DATABASE=TechStore;"
        "Trusted_Connection=yes;"
        "TrustServerCertificate=yes;"
    )
    return conexion
```

Esta configuración utiliza autenticación de Windows. Si SQL Server está instalado en otra instancia o equipo, será necesario modificar SERVER según la configuración local.

4. Instalación y ejecución

4.1 Abrir la terminal

Ingresar a la carpeta del proyecto:

```
cd "C:\Users\Usuario\TechStoreAPI"
```

4.2 Activar el entorno virtual

Si ya existe el entorno virtual venv, ejecutar:

```
.\venv\Scripts\activate
```

4.3 Instalar las dependencias

Si las bibliotecas todavía no están instaladas, ejecutar:

```
python -m pip install fastapi uvicorn pyodbc
```

4.4 Iniciar la API

Ejecutar el siguiente comando:

```
uvicorn main:app --reload
```

Cuando la aplicación se inicie correctamente, se podrá acceder a ella desde el navegador.

5. Acceso a la documentación

La API dispone de documentación interactiva mediante Swagger.

Recurso	Dirección
Documentación	http://127.0.0.1:8000/docs
Ruta principal	http://127.0.0.1:8000/

En Swagger se pueden consultar los endpoints, introducir datos de prueba, ejecutar solicitudes y revisar las respuestas HTTP.

Para probar un endpoint, se debe abrirlo, pulsar **Try it out**, ingresar los parámetros o el JSON solicitado y pulsar **Execute**.

6. Endpoints CRUD

Las siguientes cinco tablas cuentan con operaciones CRUD.

6.1 Productos

Método	Endpoint	Función
GET	/productos	Consultar todos los productos
GET	/productos/{id_producto}	Consultar un producto
POST	/productos	Crear un producto
PUT	/productos/{id_producto}	Actualizar un producto
DELETE	/productos/{id_producto}	Eliminar un producto si las relaciones lo permiten

Ejemplo para crear un producto:

```
{
  "Nombre": "Monitor Samsung",
  "Descripcion": "Monitor de prueba de 24 pulgadas",
  "IdCategoria": 2,
  "IdMarca": 1,
  "PrecioVenta": 1200.00,
  "CostoReferencia": 900.00
}
```

Los valores de IdCategoria e IdMarca deben existir en la base de datos.

6.2 Categorías

Método	Endpoint	Función
GET	/categorias	Consultar todos los categorías
GET	/categorias/{id_categoria}	Consultar una categoría
POST	/categorias	Crear una categoría
PUT	/categorias/{id_categoria}	Actualizar una categoría
DELETE	/categorias/{id_categoria}	Eliminar una categoría si las relaciones lo permiten

Ejemplo para crear una categoría:

```
{
  "Nombre": "Monitores",
  "Descripcion": "Pantallas y monitores para computadoras"
}
```

6.3 Marcas

Método	Endpoint	Función
GET	/marcas	Consultar todos los marcas
GET	/marcas/{id_marca}	Consultar una marca
POST	/marcas	Crear una marca
PUT	/marcas/{id_marca}	Actualizar una marca
DELETE	/marcas/{id_marca}	Eliminar una marca si las relaciones lo permiten

Ejemplo para crear una marca:

```
{
  "Nombre": "Samsung"
}
```

6.4 Proveedores

Método	Endpoint	Función
GET	/proveedores	Consultar todos los proveedores
GET	/proveedores/{id_proveedor}	Consultar un proveedor
POST	/proveedores	Crear un proveedor
PUT	/proveedores/{id_proveedor}	Actualizar un proveedor
DELETE	/proveedores/{id_proveedor}	Eliminar un proveedor si las relaciones lo permiten

Ejemplo para crear un proveedor:

```
{
  "Nombre": "Proveedor de prueba",
  "NIT": "1234567-8",
  "Telefono": "55551234"
}
```

6.5 Clientes

Método	Endpoint	Función
GET	/clientes	Consultar todos los clientes
GET	/clientes/{id_cliente}	Consultar un cliente
POST	/clientes	Crear un cliente
PUT	/clientes/{id_cliente}	Actualizar un cliente
DELETE	/clientes/{id_cliente}	Eliminar un cliente si las relaciones lo permiten

Ejemplo para crear un cliente:

```
{
  "Nombres": "Juan",
  "Apellidos": "Pérez",
  "NIT": "9876543-2",
  "Telefono": "55559876"
}
```

Nota. Los ejemplos de creación son datos de prueba. Antes de enviarlos, se deben comprobar las claves foráneas y evitar duplicados que infrinjan las restricciones de la base de datos.

7. Procedimientos almacenados

7.1 Registrar una venta

Endpoint: POST /operaciones/ventas

Este endpoint ejecuta el procedimiento almacenado RegistrarVenta, que registra la venta y su detalle, descuenta las existencias del inventario y registra el movimiento de salida.

Ejemplo de solicitud:

```
{
  "IdVenta": 3,
  "IdCliente": 1,
  "IdEmpleado": 1,
  "IdSucursal": 1,
  "IdDetalleVenta": 5,
  "IdProducto": 1,
  "IdAlmacen": 1,
  "Cantidad": 1,
  "PrecioUnitario": 4500.00
}
```

Los identificadores deben estar disponibles y las referencias deben existir. El inventario debe tener suficientes unidades para completar la venta.

7.2 Registrar una compra

Endpoint: POST /operaciones/compras

Este endpoint ejecuta el procedimiento almacenado RegistrarCompra, que registra la compra y su detalle, aumenta las existencias y registra el movimiento de entrada.

Ejemplo de solicitud:

```
{
  "IdCompra": 3,
  "IdProveedor": 1,
  "IdEmpleado": 1,
  "IdAlmacen": 1,
  "IdDetalleCompra": 6,
  "IdProducto": 1,
  "Cantidad": 2,
  "CostoUnitario": 3500.00
}
```

Se deben utilizar identificadores nuevos para la compra y el detalle, además de referencias válidas a proveedores, empleados, productos y almacenes.

> **Importante.** Si los identificadores de los ejemplos ya existen, deben sustituirse por otros disponibles. Los procedimientos almacenados controlan la transacción para confirmar los cambios con COMMIT o revertirlos con ROLLBACK ante un error.

8. Reportes

8.1 Ventas por sucursal

Endpoint: GET /reportes/ventas-sucursal

Devuelve la cantidad de ventas y el total vendido por sucursal. Utiliza COUNT(), SUM() y GROUP BY.

8.2 Productos más vendidos

Endpoint: GET /reportes/productos-vendidos

Devuelve las unidades vendidas y el total generado por producto. Utiliza SUM() y GROUP BY, y ordena los resultados por unidades vendidas.

Ambos reportes se pueden ejecutar desde Swagger sin enviar un cuerpo JSON.

9. Validaciones y manejo de errores

La API utiliza modelos de Pydantic para validar los datos recibidos, como campos obligatorios, longitudes de texto, identificadores positivos y valores monetarios no negativos.

Las respuestas pueden incluir los siguientes códigos HTTP:

Código	Significado
200 OK	Consulta u operación exitosa.
400 Bad Request	Operación rechazada o error en los datos o restricciones de la base de datos.
404 Not Found	Registro solicitado no encontrado.
422 Unprocessable Entity	Datos de entrada que no cumplen las validaciones de la API.
500 Internal Server Error	Error durante una consulta o ejecución interna.

Una eliminación puede fallar si el registro está relacionado con otros datos. Esto ayuda a preservar la integridad referencial.

10. Verificación de los resultados

Después de ejecutar una operación, se pueden verificar los cambios desde SQL Server Management Studio.

```
USE TechStore;

SELECT * FROM Ventas;
SELECT * FROM DetalleVentas;
SELECT * FROM Compras;
SELECT * FROM DetalleCompras;
SELECT * FROM Inventario;
SELECT * FROM MovimientosInventario;
```

Estas consultas permiten comprobar los registros creados, las modificaciones del inventario y los movimientos asociados a las operaciones.

11. Detener la API

Para detener el servidor, regresar a la terminal donde se está ejecutando Uvicorn y presionar:

```
Ctrl + C
```